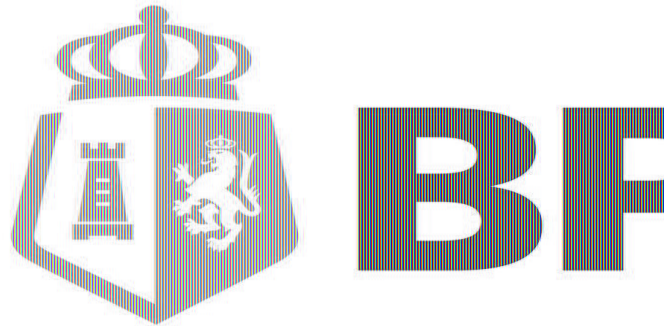




Java Exception Handling & Logging with SLF4J + Logback

A BPI Training Program

Welcome to Part 2A - Logging with SLF4J!

PART 2A: Logging with SLF4J

Part 2A - Logging with SLF4!

The Golden Rule:

"If it's not logged, it didn't happen"

- Every developer

Think about it:

If a problem happens but nobody recorded it...

How will you fix it?

How will you prove it happened?

How will you prevent it next time?

Logging = Recording what happens in your program!

What is SLF4J?

SLF4J = Simple Logging Facade for Java

Simple Explanation:

SLF4J is an interface that lets you write logging code that works with ANY logging library!

Think of it as a Universal Remote:

🎮 Remote (SLF4J) → Works with any TV (Logging Library)

Why SLF4J?

- ✓ Write code once, use any logging library
- ✓ Industry standard
- ✓ Easy to use

The Problem - System.out.println

Why System.out.println is NOT Enough

What beginners do:

```
System.out.println("User logged in");  
System.out.println("ERROR: Database failed!");  
System.out.println("Withdrawal successful");
```

What's wrong with this?

- ✗ No timestamp - when did it happen?
- ✗ All messages look the same - can't tell what's important
- ✗ Can't save to file automatically
- ✗ Can't turn off in production
- ✗ Can't search through messages easily
- ✗ No severity levels - error looks same as normal message
- ✗ NOT professional!

What is Professional Logging?

Professional Logging - What You Need

A good log message has:

- Timestamp - exactly when it happened
- Severity level - how important is it?
- Category - which part of the system?
- Clear message - what happened?
- Context - relevant details

See the difference?

- We know WHEN (2025-01-15 14:30:25)
- We know HOW SERIOUS (ERROR)
- We know WHERE (BankAccount class)
- We know WHAT (Withdrawal failed, why, and details)

Example of a good log:

```
[2025-01-15 14:30:25] [ERROR] [BankAccount] Withdrawal failed:  
Insufficient funds for account ACC-12345. Balance: ₱5,000.0, Requested:  
₱10,000.0
```



Log Levels - How Important Is This Message?

Log Levels - Severity Scale

From least to most serious:

TRACE 🔍

- Very detailed, fine-grained logging
- Used to trace program flow (method entry/exit, variable values)
- Rarely enabled in production
- **Example:** "Entering calculateInterest() method"

DEBUG 🔍

- Detailed information for diagnosing problems
- Only needed during development
- Example: "Checking balance for account ACC-123"

INFO ⓘ

- Normal events, everything is okay
- General information
- Example: "User John logged in successfully"

WARN ⚠️

- Something unusual, but not an error
- Potential problem
- Example: "Withdrawal of ₱50,000 - unusually high amount"

ERROR ❌

- Something went wrong!
- Actual errors that need attention
- Example: "Database connection failed!"

How to Use Logging - The Syntax

How to Write Logging Code

Step 1: Import the classes

```
import org.slf4j.Logger;  
import org.slf4j.LoggerFactory;
```

Step 2: Create a logger (at class level)

```
private static final Logger logger =  
    LoggerFactory.getLogger(ClassName.class);
```

Step 3: Use logger methods

```
logger.debug("Debug message");  
logger.info("Information message");  
logger.warn("Warning message");  
logger.error("Error message");
```

Step 4: Include values with {} placeholders

```
logger.info("User {} logged in", username);  
logger.warn("Balance low: P{}", balance);  
logger.error("Failed to process P{} for  
account {}", amount, accountNumber);
```

Important:

Use {}, not + for concatenation!



Code Demo

SEATWORK #6: Your Turn to Try! (Branch: M4_Activity6)

Instructions:

1. Create Custom Exceptions

- `InvalidAmountException` - extends `Exception`, has `String` message constructor
- `InsufficientFundsException` - extends `Exception`, has `message` + `balance` + `requestedAmount` fields with getters

2. Create Functional Interface

- `BankTestOperation` with `@FunctionalInterface` annotation
- Single method: `execute()` throws `InvalidAmountException`, `InsufficientFundsException`

3. Create BankAccount Class

- Logger: `LoggerFactory.getLogger(M4Activity6.class)`
- Initial balance: ₱10,000

4. Implement `withdraw(double amount)` - throws both exceptions

- Log INFO: "Withdrawal requested: ₱{"}
- If `amount < 0` → Log ERROR, throw `InvalidAmountException`
- If `amount > balance` → Log WARN, throw `InsufficientFundsException`
- Process: `balance -= amount`
- Log INFO: "Withdrawal completed: ₱{"}, New balance: ₱{"}

5. Implement `deposit(double amount)` - throws `InvalidAmountException`

- Log INFO: "Deposit requested: ₱{"}
- If `amount <= 0` → Log ERROR, throw `InvalidAmountException`
- If `amount > 50000` → Log WARN (but continue processing)
- Process: `balance += amount`
- Log INFO: "Deposit completed: ₱{"}, New balance: ₱{"}

6. Implement `runTest()` - static helper method

- Parameters: `BankTestOperation operation`, `String operationName`
- Try to execute operation
- Catch `InvalidAmountException` → Log ERROR with exception object
- Catch `InsufficientFundsException` → Log ERROR with exception object

7. Test in `main()` - Use `runTest()` with lambda expressions

- Test 1: `runTest() -> account.deposit(5000), "Deposit"`
- Test 2: `runTest() -> account.withdraw(3000), "Withdrawal"`
- Test 3: `runTest() -> account.deposit(-500), "Deposit"`
- Test 4: `runTest() -> account.withdraw(20000), "Withdrawal"`
- Test 5: `runTest() -> account.deposit(60000), "Deposit"`

SEATWORK #6: Your Turn to Try! (Branch: M4_Activity6)

Expected Output:

```
01:14:25.209 [INFO] com.bpi.training.lecture.activity.BankAccount - Deposit requested: P5000.0
01:14:25.217 [INFO] com.bpi.training.lecture.activity.BankAccount - Deposit completed: P5000.0, New balance: P15000.0
01:14:25.218 [INFO] com.bpi.training.lecture.activity.BankAccount - Withdrawal requested: P3000.0
01:14:25.218 [INFO] com.bpi.training.lecture.activity.BankAccount - Withdrawal completed: P3000.0, New balance: P12000.0
01:14:25.219 [INFO] com.bpi.training.lecture.activity.BankAccount - Deposit requested: P-500.0
01:14:25.219 [ERROR] com.bpi.training.lecture.activity.BankAccount - Invalid deposit amount: P-500.0
01:14:25.219 [ERROR] com.bpi.training.lecture.activity.BankAccount - Deposit failed: Deposit amount must be positive
com.bpi.training.lecture.activity.exception.InvalidAmountException: Deposit amount must be positive
    at com.bpi.training.lecture.activity.BankAccount.deposit(BankAccount.java:39)
    at com.bpi.training.lecture.activity.BankAccount.lambda$2(BankAccount.java:72)
    at com.bpi.training.lecture.activity.BankAccount.runTest(BankAccount.java:53)
    at com.bpi.training.lecture.activity.BankAccount.main(BankAccount.java:72)
01:14:25.222 [INFO] com.bpi.training.lecture.activity.BankAccount - Withdrawal requested: P20000.0
01:14:25.223 [WARN] com.bpi.training.lecture.activity.BankAccount - Insufficient funds: P12000.0 available
01:14:25.223 [ERROR] com.bpi.training.lecture.activity.BankAccount - Withdrawal failed: Insufficient funds for withdrawal
com.bpi.training.lecture.activity.exception.InsufficientFundsException: Insufficient funds for withdrawal
    at com.bpi.training.lecture.activity.BankAccount.withdraw(BankAccount.java:25)
    at com.bpi.training.lecture.activity.BankAccount.lambda$3(BankAccount.java:75)
    at com.bpi.training.lecture.activity.BankAccount.runTest(BankAccount.java:53)
    at com.bpi.training.lecture.activity.BankAccount.main(BankAccount.java:75)
01:14:25.224 [INFO] com.bpi.training.lecture.activity.BankAccount - Deposit requested: P60000.0
01:14:25.224 [WARN] com.bpi.training.lecture.activity.BankAccount - Large deposit: P60000.0 - requires verification
01:14:25.224 [INFO] com.bpi.training.lecture.activity.BankAccount - Deposit completed: P60000.0, New balance: P72000.0
```

Welcome to Part 2B - Logging with SLF4J + Logback!

PART 2B: Logging with SLF4J + Logback

Setting Up SLF4J + Logback

Now you know HOW to use logging in code!

But how do we SET IT UP?

What we'll learn:

1. Understand what Logback is
2. Create configuration file (logback.xml)
3. Configure where logs go (console/file)
4. Test everything works!

Let's make logging work in YOUR projects!

What is Logback?

Definition: Logback is a Java library that handles your logging automatically.

What you write:

```
→ logger.info("Message");
```

What Logback creates:

```
→ 2025-01-15 14:30:25 [INFO] BankAccount - Message
```

Logback does 3 things:

1. Formats your messages (adds time, level, class name)
2. Sends logs where you want (console, file, database)
3. Manages log files (creates, rotates, deletes old logs)

Why Logback?

✓ Industry Standard

- Used by banks, tech companies, e-commerce worldwide
- Trusted by millions of developers

✓ Flexible & Powerful

- Change logging behavior without code changes
- Just edit configuration file!

✓ Professional Features

- Automatic log file rotation (daily, hourly)
- Automatic compression (zip old logs)
- Automatic cleanup (delete very old logs)

✓ Fast Performance

- Handles thousands of logs per second
- Won't slow down your application

✓ Easy to Use

- Simple XML configuration
- Works seamlessly with SLF4J

Create Basic logback.xml Configuration

Create Basic Configuration

File Location: **src/logback.xml**

Basic logback.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>

    <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>%d{HH:mm:ss.SSS} [%level] %logger - %msg%n</pattern>
        </encoder>
    </appender>

    <root level="INFO">
        <appender-ref ref="CONSOLE"/>
    </root>

</configuration>
```




Code Demo

Components of logback.xml

logback.xml has 3 main components:

1. APPENDER

- WHERE do logs go?
- Console, File, Database, Email?

2. LAYOUT (Encoder + Pattern)

- HOW do logs look?
- Format: Time, Level, Message

3. ROOT

- WHAT level to show?
- WHICH appenders to use?

Component 1 - Appender (WHERE)

What is an Appender?

- Component that sends logs to a destination

Structure:

```
<appender name="YOUR_NAME" class="APPENDER_TYPE">  
  <encoder>  
    <pattern>FORMAT</pattern>  
  </encoder>  
</appender>
```

Attributes:

name = "YOUR_NAME"

- Your label (choose any name)
- Examples: "CONSOLE", "FILE", "DAILY_LOG"
- Use this name later in <appender-ref>

class = "APPENDER_TYPE"

- Type of output destination - Full Java class name

Common Appender Types:

1. ConsoleAppender

```
class="ch.qos.logback.core.ConsoleAppender"
```

- Writes to console (STS4 console window)
- Good for: Development

2. FileAppender

```
class="ch.qos.logback.core.FileAppender"
```

- Writes to a single file
- Good for: Simple production logging
- `<file></file>`
 - Tells Logback WHERE to save the log file

3. RollingFileAppender (Advanced!)

```
class="ch.qos.logback.core.rolling.RollingFileAppender"
```

- Creates new files automatically (daily, hourly, by size)
- Good for: Production systems with lots of logs

Example - Console Appender:

```
<appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
  <encoder>
    <pattern>%d{HH:mm:ss} [%level] - %msg%n</pattern>
  </encoder>
</appender>
```

Example - File Appender:

```
<appender name="FILE" class="ch.qos.logback.core.FileAppender">
  <file>logs/app.log</file>
  <encoder>
    <pattern>%d{yyyy-MM-dd HH:mm:ss} [%level] - %msg%n</pattern>
  </encoder>
</appender>
```

Component 2 - Layout (Encoder & Pattern)

Component 2: Layout - HOW Logs Look

Layout = Encoder + Pattern

Structure:

```
<encoder>  
    <pattern>FORMAT_TEMPLATE</pattern>  
</encoder>
```

Encoder:

- Converts log events to text
- Standard: PatternLayoutEncoder (default)



Pattern:

- Template with special codes
- Codes start with %

Common Pattern Codes:

%d or %date - Date/Time 

%d → 2025-01-15 14:30:25,123

%d{HH:mm:ss} → 14:30:25

%d{HH:mm:ss.SSS} → 14:30:25.123 (*Good for console*)

%d{yyyy-MM-dd HH:mm:ss} → 2025-01-15 14:30:25 (*Good for files*)

%level or %le or %p - Log Level

%level → INFO, WARN, ERROR, DEBUG

[%level] → [INFO], [WARN], [ERROR] (with brackets)

%-5level → INFO (fixed 5-char width, padded)

%logger or %c - Logger Name

%logger → com.banking.training.BankAccount

%logger{36} → com.banking.training.BankAccount (max 36 chars)

%logger{0} → BankAccount (class name only)

%logger{5} → c.b.t.BankAccount (abbreviated)

%msg or %m - Your Message

%msg → The actual message you wrote

%n - New Line ↵

%n → Platform-independent line break → Always end pattern with %n!

%thread or %t - Thread Name 🧵

%thread → main, Thread-1, pool-2-thread-3

%F - File Name 📄

%F → BankAccount.java

%L - Line Number 📄

%L → 45

%M - Method Name 🔧

%M → withdraw

Pattern Examples:

Simple (Console):

```
%d{HH:mm:ss} [%level] - %msg%n
```

Output: 14:30:25 [INFO] - Withdrawal successful

Standard (Console):

```
%d{HH:mm:ss.SSS} [%level] %logger{36} - %msg%n
```

Output: 14:30:25.123 [INFO] ...BankAccount - Withdrawal successful

Production (File):

```
%d{yyyy-MM-dd HH:mm:ss.SSS} [%level] %logger{36} - %msg%n
```

Output: 2025-01-15 14:30:25.123 [INFO] ...BankAccount - Withdrawal successful

Detailed (Debugging):

```
%d{HH:mm:ss.SSS} [%level] %logger.%M:%L - %msg%n
```

Output: 14:30:25.123 [INFO] ...BankAccount.withdraw:45 - Withdrawal successful



Component 3 - Root (WHAT & WHICH)

Root - WHAT to Show & WHICH Appenders

Structure:

```
<root level="LOG_LEVEL">  
  <appender-ref ref="APPENDER_NAME"/>  
</root>
```

Root controls 2 things:

- 1. level="LOG_LEVEL" - Minimum log level to show
- 2. <appender-ref ref="APPENDER_NAME"/>
 - Which appenders to use
 - Can have multiple!

Log Levels (Hierarchy):

- DEBUG - Show ALL messages (DEBUG, INFO, WARN, ERROR)
- INFO - Hide DEBUG (INFO, WARN, ERROR)
- WARN - Hide DEBUG + INFO (WARN, ERROR)
- ERROR - Only errors (ERROR only)
- OFF - No logging



When to use each level:

- TRACE - Very detailed flow tracing
- DEBUG - Development, detailed debugging
- INFO - Production (normal operations)
- WARN - Production (investigating issues)
- ERROR - Emergency (minimal logging)
- OFF - Disable logging completely

Examples:

One Appender:

```
<root level="INFO">
  <appender-ref ref="CONSOLE"/>
</root>
```

- INFO, WARN, ERROR to console

Multiple Appenders:

```
<root level="INFO">
  <appender-ref ref="CONSOLE"/>
  <appender-ref ref="FILE"/>
</root>
```

- INFO, WARN, ERROR to BOTH console AND file

Different Level:

```
<root level="DEBUG">
  <appender-ref ref="CONSOLE"/>
</root>
```

- ALL messages (DEBUG, INFO, WARN, ERROR) to console

Production Setup:

```
<root level="WARN">
  <appender-ref ref="FILE"/>
</root>
```

- Only WARN and ERROR to file

Enable TRACE:

```
<root level="TRACE">
  <appender-ref ref="CONSOLE"/>
</root>
```

- ALL messages (TRACE, DEBUG, INFO, WARN, ERROR) to console

Configure Console & File Logging

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>

  <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
      <pattern>%d{HH:mm:ss.SSS} [%level] %logger{36} - %msg%n</pattern>
    </encoder>
  </appender>

  <appender name="FILE" class="ch.qos.logback.core.FileAppender">
    <file>logs/application.log</file>
    <encoder>
      <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%level] %logger{36} - %msg%n</pattern>
    </encoder>
  </appender>

  <root level="INFO">
    <appender-ref ref="CONSOLE"/>
    <appender-ref ref="FILE"/>
  </root>

</configuration>
```



Code Demo

SEATWORK #7: Your Turn to Try! (Branch: M4_Activity7)

Time: 10 minutes

Your Task: Add dual logging (console + file) to your BankAccount class

Instructions:

1. Update your logback.xml to have:

- Console appender (pattern with SHORT time: HH:mm:ss.SSS)
- File appender (pattern with FULL date: yyyy-MM-dd HH:mm:ss.SSS)
- File location: **logs/application.log**
- Both appenders referenced in root
- Root level: INFO

SEATWORK #7: Your Turn to Try! (Branch: M4_Activity7)

Expected Output:

```
01:14:25.209 [INFO] com.bpi.training.lecture.activity.BankAccount - Deposit requested: ₱5000.0
01:14:25.217 [INFO] com.bpi.training.lecture.activity.BankAccount - Deposit completed: ₱5000.0, New balance: ₱15000.0
01:14:25.218 [INFO] com.bpi.training.lecture.activity.BankAccount - Withdrawal requested: ₱3000.0
01:14:25.218 [INFO] com.bpi.training.lecture.activity.BankAccount - Withdrawal completed: ₱3000.0, New balance: ₱12000.0
01:14:25.219 [INFO] com.bpi.training.lecture.activity.BankAccount - Deposit requested: ₱-500.0
01:14:25.219 [ERROR] com.bpi.training.lecture.activity.BankAccount - Invalid deposit amount: ₱-500.0
01:14:25.219 [ERROR] com.bpi.training.lecture.activity.BankAccount - Deposit failed: Deposit amount must be positive
com.bpi.training.lecture.activity.exception.InvalidAmountException: Deposit amount must be positive
    at com.bpi.training.lecture.activity.BankAccount.deposit(BankAccount.java:39)
    at com.bpi.training.lecture.activity.BankAccount.lambda$2(BankAccount.java:72)
    at com.bpi.training.lecture.activity.BankAccount.runTest(BankAccount.java:53)
    at com.bpi.training.lecture.activity.BankAccount.main(BankAccount.java:72)
01:14:25.222 [INFO] com.bpi.training.lecture.activity.BankAccount - Withdrawal requested: ₱20000.0
01:14:25.223 [WARN] com.bpi.training.lecture.activity.BankAccount - Insufficient funds: ₱12000.0 available
01:14:25.223 [ERROR] com.bpi.training.lecture.activity.BankAccount - Withdrawal failed: Insufficient funds for withdrawal
com.bpi.training.lecture.activity.exception.InsufficientFundsException: Insufficient funds for withdrawal
    at com.bpi.training.lecture.activity.BankAccount.withdraw(BankAccount.java:25)
    at com.bpi.training.lecture.activity.BankAccount.lambda$3(BankAccount.java:75)
    at com.bpi.training.lecture.activity.BankAccount.runTest(BankAccount.java:53)
    at com.bpi.training.lecture.activity.BankAccount.main(BankAccount.java:75)
01:14:25.224 [INFO] com.bpi.training.lecture.activity.BankAccount - Deposit requested: ₱60000.0
01:14:25.224 [WARN] com.bpi.training.lecture.activity.BankAccount - Large deposit: ₱60000.0 - requires verification
01:14:25.224 [INFO] com.bpi.training.lecture.activity.BankAccount - Deposit completed: ₱60000.0, New balance: ₱72000.0
```


Advanced - Rolling File Appender

Advanced: Rolling File Appender (Daily/Hourly Logs)

Problem with FileAppender:

- Single file keeps growing forever
- Gets huge (10MB, 100MB, 1GB!)
- Hard to find specific logs
- Hard to manage

Solution: RollingFileAppender

- Creates NEW files automatically
- By time: Daily, hourly, monthly
- By size: Every 10MB, 50MB, etc.
- Compresses old files (.gz)
- Deletes very old files

Common Rolling Policy XML Tags Reference

1. `<rollingPolicy class="...">`
 - Container for rolling configuration

Common Classes:

- `ch.qos.logback.core.rolling.TimeBasedRollingPolicy` → Roll by time (daily, hourly, monthly, etc.)
- `ch.qos.logback.core.rolling.SizeAndTimeBasedRollingPolicy` → Roll by BOTH time AND size



Common Rolling Policy XML Tags Reference

2. <fileNamePattern>
- Pattern for archived file names

Common Date/Time Patterns:

%d{ yyyy-MM-dd }	→ Daily:	app-2025-01-15.log
%d{ yyyy-MM-dd-HH }	→ Hourly:	app-2025-01-15-14.log
%d{ yyyy-MM }	→ Monthly:	app-2025-01.log
%d{ yyyy-ww }	→ Weekly:	app-2025-03.log

Size-Based Pattern:

%i → Index: app-2025-01-15-0.log, app-2025-01-15-1.log

Compression:

.gz	→ GZIP:	app-2025-01-15.log.gz
.zip	→ ZIP:	app-2025-01-15.log.zip

Examples:

```
<fileNamePattern>logs/app-%d{ yyyy-MM-dd }.log</fileNamePattern>
<fileNamePattern>logs/app-%d{ yyyy-MM-dd }.log.gz</fileNamePattern>
<fileNamePattern>logs/app-%d{ yyyy-MM-dd }-%i.log</fileNamePattern>
```



Common Rolling Policy XML Tags Reference

3. `<maxHistory>`
- Number of archived files to keep (older files deleted)

Unit depends on fileNamePattern:

Daily pattern	→ <code><maxHistory>30</maxHistory></code>	→ Keep 30 days
Hourly pattern	→ <code><maxHistory>168</maxHistory></code>	→ Keep 168 hours (7 days)
Monthly pattern	→ <code><maxHistory>12</maxHistory></code>	→ Keep 12 months (1 year)

Examples:

<code><maxHistory>7</maxHistory></code>	→ 1 week
<code><maxHistory>30</maxHistory></code>	→ 30 days
<code><maxHistory>90</maxHistory></code>	→ 90 days



Common Rolling Policy XML Tags Reference

4. <totalSizeCap>
- Maximum total disk space for all log files

Purpose: Limit total disk space used by logs
When exceeded: Oldest files deleted automatically

Size Units:

KB → Kilobytes
MB → Megabytes
GB → Gigabytes

Examples:

```
<totalSizeCap>100MB</totalSizeCap>  
<totalSizeCap>1GB</totalSizeCap>  
<totalSizeCap>10GB</totalSizeCap>
```

5. <maxFileSize>
- Maximum size of each log file (for size-based rolling)

Purpose: Create new file when current file reaches this size
Requires: SizeAndTimeBasedRollingPolicy
Requires: %i in fileNamePattern

Size Units:

KB → Kilobytes
MB → Megabytes
GB → Gigabytes

Examples:

```
<maxFileSize>10MB</maxFileSize>  
<maxFileSize>50MB</maxFileSize>  
<maxFileSize>100MB</maxFileSize>
```

Example: Daily Rolling File

```
<appender name="DAILY_FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">

  <!-- Current log file -->
  <file>logs/application.log</file>

  <!-- Rolling policy: Create new file daily -->
  <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">

    <!-- File name pattern: YYYY-MM-DD -->
    <fileNamePattern>logs/application-%d{yyyy-MM-dd}.log</fileNamePattern>

    <!-- Keep 30 days of logs -->
    <maxHistory>30</maxHistory>

    <!-- Max total size: 1GB -->
    <totalSizeCap>1GB</totalSizeCap>

  </rollingPolicy>

  <!-- Log format -->
  <encoder>
    <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%level] %logger{36} - %msg%n</pattern>
  </encoder>
</appender>
```



How it works:

- Day 1 (Dec 1):
- Current: logs/application.log
- Day 2 (Dec 2) at midnight:
- Renamed: logs/application-2025-12-01.log
 - New current: logs/application.log
- Day 3 (Dec 3) at midnight:
- Renamed: logs/application-2025-12-02.log
 - New current: logs/application.log
- After 30 days:
- Oldest file deleted automatically!

Result:

- | | |
|----------------------------|-------------------|
| logs/ | |
| application.log | ← Current (today) |
| application-2025-12-01.log | ← Yesterday |
| application-2025-12-02.log | ← 2 days ago |
| application-2025-12-03.log | ← 3 days ago |
| ... | ← Up to 30 days |



More Rolling File Examples

Example 1: Hourly Rolling

```
<appender name="HOURLY_FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
  <file>logs/application.log</file>
  <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
    <fileNamePattern>logs/application-%d{yyyy-MM-dd-HH}.log</fileNamePattern>
    <maxHistory>168</maxHistory> <!-- 7 days × 24 hours -->
  </rollingPolicy>
  <encoder>
    <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%level] %logger - %msg%n</pattern>
  </encoder>
</appender>
```

Result: New file every hour

- logs/application-2025-01-15-14.log (2 PM)
- logs/application-2025-01-15-15.log (3 PM)
- logs/application-2025-01-15-16.log (4 PM)

More Rolling File Examples

Example 2: Size-Based Rolling

```
<appender name="SIZE_FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
  <file>logs/application.log</file>
  <rollingPolicy class="ch.qos.logback.core.rolling.SizeAndTimeBasedRollingPolicy">
    <fileNamePattern>logs/application-%d{yyyy-MM-dd}-%i.log</fileNamePattern>
    <maxFileSize>10MB</maxFileSize>
    <maxHistory>30</maxHistory>
    <totalSizeCap>500MB</totalSizeCap>
  </rollingPolicy>
  <encoder>
    <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%level] %logger - %msg%n</pattern>
  </encoder>
</appender>
```

Result: New file when size reaches 10MB

- logs/application-2025-01-15-0.log (first 10MB)
- logs/application-2025-01-15-1.log (second 10MB)
- logs/application-2025-01-15-2.log (third 10MB)

More Rolling File Examples

Example 3: Compressed Archives

```
<appender name="COMPRESSED_FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
  <file>logs/application.log</file>
  <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
    <fileNamePattern>logs/application-%d{yyyy-MM-dd}.log.gz</fileNamePattern>
    <maxHistory>30</maxHistory>
  </rollingPolicy>
  <encoder>
    <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%level] %logger - %msg%n</pattern>
  </encoder>
</appender>
```

Note: .gz at end = automatic compression!
Result: Old files automatically compressed to save space

- logs/application-2025-12-01.log.gz (compressed)
- logs/application-2025-12-02.log.gz (compressed)



Code Demo

SEATWORK #8: Your Turn to Try! (Branch: M4_Activity8)

Time: 10 minutes

Your Task: Upgrade BankAccount logging with daily rolling file appender

Instructions:

1. Update your logback.xml to have:

- Console appender
- Daily Rolling File appender (NEW!)
 - Current file: logs/bank-app.log
 - Archive pattern: logs/bank-app-%d{yyyy-MM-dd}.log
 - Keep 30 days of logs
 - Max total size: 1GB
- Both referenced in root
- Root level: INFO